A visual guide to the proof of the $\beta\eta$-Barendregt–Geuvers–Klop conjectures

Ankit Pradhan Cole Vick Ruijie Fang Carl Kwan

The University of Texas at Austin

October 27, 2026

Abstract

The Barendregt–Geuvers–Klop conjecture states that a pure type system in which every legal term is weakly normalizing is also strongly normalizing. OpenAI recently proved its β instance for systems with β-conversion. We extend that proof to Geuvers's systems $\mathrm{PTS}_{\beta\eta}$, whose conversion rule uses $\beta\eta$-conversion, and formalize the extension in Lean. Both proofs are long. The Lean formalization of the $\beta\eta$ case has about 48 000 lines, of which about 30 000 are OpenAI's. This paper is a visual guide to the two proofs. It first draws the thirty-year history of the conjecture, and shows how close earlier translation methods came and why they stopped at dependent systems. It then presents OpenAI's proof as a single recursion over the strongly connected components of a finite signed graph on sets of sorts, and illustrates each construction of the recursion on a small example: the signed fixed point, the recursion along one path, the observation values that interpret variables, and the typed realization of Hurkens's paradox that excludes one graph configuration. Next it asks where a shorter proof might come from. Two small specifications show that the three constructions of the candidate argument cannot be merged, and the standard systems never need the uniform stage bounds of the third construction. Counts over random specifications suggest that a generic form of Hurkens's paradox would cover only about half of the configurations that the paradox must exclude. Finally, the guide turns to the $\beta\eta$ proof, which runs the same recursion. It locates the four places where η breaks the β proof and presents the three repairs: reducts that are normal outside annotations, keys computed from skeletons, and η-canonical observation values. Every figure names the Lean files and declarations it depicts.

Contents

1 Introduction **2**

2 History **3**

3 The β proof **6**

4 Candidates and why they are hard to build **7**

5 The three cases **8**

6 The forbidden configuration on familiar systems **11**

7 A shorter proof? **13**

8 The $\beta\eta$ proof **15**

9 What η changes **16**

1 Introduction

A pure type system is specified by a set of sorts, an axiom relation and a rule relation [1]. Its legal terms are those that occur in derivable typing judgments. The system is weakly normalizing if every legal term has a normal form, and strongly normalizing if no legal term admits an infinite reduction. The Barendregt–Geuvers–Klop conjecture asserts that weak normalization implies strong normalization [18, Conjecture 8.1.2], [40]. Geuvers states it for β- and for $\beta\eta$-reduction, and for pure type systems whose conversion rule uses β or $\beta\eta$.

The two proofs. OpenAI proved the β instance for the systems PTS_β, for every specification and without any functionality assumption, and formalized the proof in Lean [35]. The argument has two halves. The first is a candidate argument in the tradition of Tait and Girard [21] that works under a finite graph condition, and the second is a typed realization of Hurkens's paradox [23, 15] that derives this condition from weak normalization. We extend the argument to the systems $\mathrm{PTS}_{\beta\eta}$ and formalize the extension in Lean [36, 13, 45]. The paradox is reused unchanged, while the candidate half is redone with new indices and canonical values.

The $\beta\eta$ result. Fix a specification, that is, an arbitrary set of sorts with axioms and rules, with no functionality, finiteness or decidability assumption. A term is legal if, in some valid context of $\mathrm{PTS}_{\beta\eta}$, it has a type or is a type.

1. If every legal term is weakly $\beta\eta$-normalizing, then every legal term is strongly $\beta\eta$-normalizing.

   `weak_implies_strong_betaEta`

2. If every legal term is weakly β-normalizing, then every legal term is strongly $\beta\eta$-normalizing.

   `weak_beta_implies_strong_betaEta`

The two Lean theorems are stated in `Challenge.lean` and proved in `Solution.lean`. Both are derived from an intermediate theorem, `strong_beta_of_eNormalizing_BE` in `Main.lean`, whose proof is drawn in Section 8.

Why a visual guide. Both proofs are long, and their structure is hard to see from the text alone. The candidate half consists of three different constructions, and the paradox accounts for more than half of OpenAI's Lean code. Yet the whole argument is organized around a single equation for candidates and a single finite graph on sets of sorts. This guide draws that structure. Each figure conveys one idea, illustrated where possible on a concrete term or specification, and names the Lean files and declarations that implement it, so that the guide also serves as a map of the two formalizations.

Main observations. Besides the pictures, the guide makes the following observations.

- Every candidate construction solves Girard's equation for the candidate of a product. Plain recursion on types fails for three reasons: circular definitions with mixed signs, substitutions that enlarge types, and variables that turn into types (Section 4).
- The case split asks the same two questions about a dependency graph twice, first for candidates and then for the parameter spaces introduced by the third case. If the signs are consistent, the proof takes a fixed point, and if some measure decreases, it uses recursion (Section 5).
- Both answers are negative only in the forbidden configuration, a layer of types that has variables and can quantify over itself. This is the shape of λU^-, and the paradox rules it out (Section 6).
- Two small specifications show that the first two cases cannot be absorbed into the third. The free minus layers, which are what makes the uniform bounds of the third case necessary, occur in no standard system and rarely in random ones (Section 7).
- The paradox makes up 55% of OpenAI's Lean code. Its relational layer is needed because β at the data level holds only up to logical equivalence. A generic form of Hurkens's paradox with exact data-level β would apply to roughly half of the random configurations. It could therefore provide a second route, but not a replacement (Section 7).
- The $\beta\eta$ proof runs the same recursion from a weaker hypothesis, namely that every legal term has a reduct that is normal outside its λ-annotations. It names types by keys, the $\beta\eta$-normal forms of terms with erased annotations, and its only new idea is to make the observation values of the third case η-canonical (Sections 8 and 9).

Scope. This guide is not a proof in its own right. The formal proofs are the Lean files cited in the figures: OpenAI's formalization of the β case [35], and the development `PTS-beta-eta` for the $\beta\eta$ case [36], which includes a copy of OpenAI's files in `PTSBetaEta/Beta`. Statements about the β proof are taken from OpenAI's preprint, and lemma

numbers such as "Lemma 31" refer to it. The development covers neither the system $\mathrm{PTS}^{s}_{\beta\eta}$ with a strengthening rule nor Geuvers's Conjecture 8.1.3, which states that strong normalization of PTS_{β} implies strong normalization of $\mathrm{PTS}_{\beta\eta}$. The counts of graph configurations over random finite specifications in Section 7 are not proofs, and that section proposes directions rather than results.

Reading the figures. Colors are used consistently. Gray marks ordinary metatheory of pure type systems, blue the candidate method, green the two simple constructions, orange the third construction and red the paradox. Violet marks material that appears only in the $\beta\eta$ proof. In profile graphs, red arrows are domain edges, which are negative, and blue arrows are codomain or inclusion edges, which are positive. Typewriter font denotes Lean files and declarations, and a number after a file name, where present, is its length in lines.

Organization. Section 2 traces the history of the conjecture and how close earlier work came to it. Sections 3 to 7 cover OpenAI's proof of the β case. Section 3 gives its overall structure, Section 4 defines candidates and explains the three obstacles to constructing them, and Section 5 presents the three cases of the construction. Section 6 shows the forbidden configuration on familiar systems and walks through the paradox, and Section 7 asks which parts of the proof are necessary and where a shorter proof might come from. Sections 8 and 9 present the $\beta\eta$ proof and what η changes.

2 History

The conjecture has a thirty-year history, and its proof draws on lines of work that long developed separately. Figure 1 shows these lines and how they influenced each other, Figure 2 places each result according to what it assumes and which systems it covers, and Figure 3 compares the mechanism of the strongest earlier results with that of OpenAI's proof.

The five lines. The first column of Figure 1 is untyped reduction without erasure. Church and Rosser showed that in the λI-calculus every term with a normal form is strongly normalizing [8]. Barendregt, Bergstra, Klop and Volken, and later Klop and Bergstra, extended this conservation property to the λK-calculus and to orthogonal systems by means of perpetual redexes [2, 28, 7]. The second column derives strong from weak normalization by translating terms into a fragment without erasure. The early methods required weak normalization of a richer reduction [34, 28, 24, 27, 12, 26], whereas Sørensen, Xi and Loader required only weak normalization of the same reduction, for particular calculi [41, 48, 29]. Barthe, Hatcliff and Sørensen obtained the result for a uniform class of non-dependent pure type systems [5, 6]. The third column proves strong normalization directly with reducibility candidates [42, 20]. This method applies only to systems that are already known to normalize, and Melliès and Werner gave a generic version for pure type systems [31]. The fourth column consists of the paradoxes that make some systems non-normalizing, from Girard's paradox in λU [20] to Hurkens's short paradox in λU^{-} [23] and Geuvers's presentation of it as a derivation [15]. The last column concerns $\beta\eta$-conversion: Nederpelt observed that $\beta\eta$ is not confluent on annotated terms [34], and Geuvers proved confluence on legal terms [14, 18].

Two facts organize this picture. First, in the λI-calculus, where no argument is ever erased, a term with a normal form is strongly normalizing [8]. All earlier results that derive strong from weak normalization rely on this fact, by translating terms into a fragment without erasure. Second, the translation must produce a typable term, which requires particular product rules. It is this requirement that stopped the translation approach at non-dependent systems. OpenAI's proof uses no translation at all. Instead it combines the reducibility method with the paradoxes that make some systems non-normalizing, two lines of work that had never been brought together.

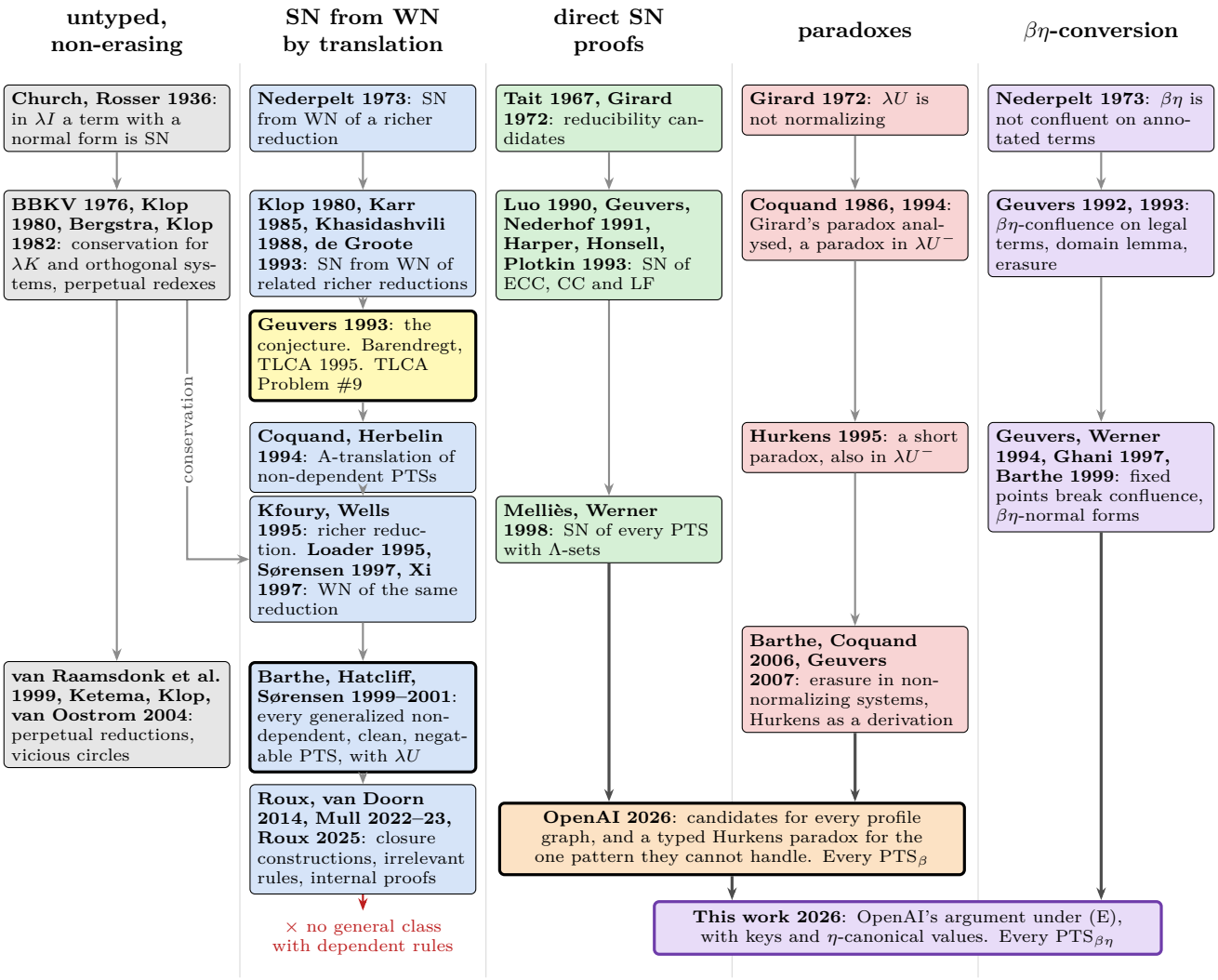


Figure 1: A genealogy of the proof. Columns are lines of work, rows run roughly forward in time, and arrows indicate the use of earlier ideas. The translation line (blue) derived SN from WN, but only up to non-dependent systems. OpenAI's proof instead combines the reducibility method (green) with the paradoxes (red). Sources: [8, 2, 28, 7, 47, 25, 34, 24, 27, 12, 18, 40, 11, 26, 29, 41, 48, 5, 6, 38, 32, 33, 37, 42, 20, 30, 16, 22, 31, 9, 10, 23, 4, 15, 14, 17, 19, 3, 35, 36].

How close each line came. Figure 2 measures the distance to the conjecture along two axes. The vertical axis is the hypothesis. The early techniques required weak normalization of a more complicated reduction, and hence a new normalization proof for each calculus [39, Ch. 2]. From Sørensen onwards, the hypothesis is that of the conjecture. The horizontal axis is the scope. Barthe, Hatcliff and Sørensen covered a uniform class that includes non-normalizing systems such as λU, and later work extended it slightly [33, 38]. None of these results reached a general class with dependent rules.

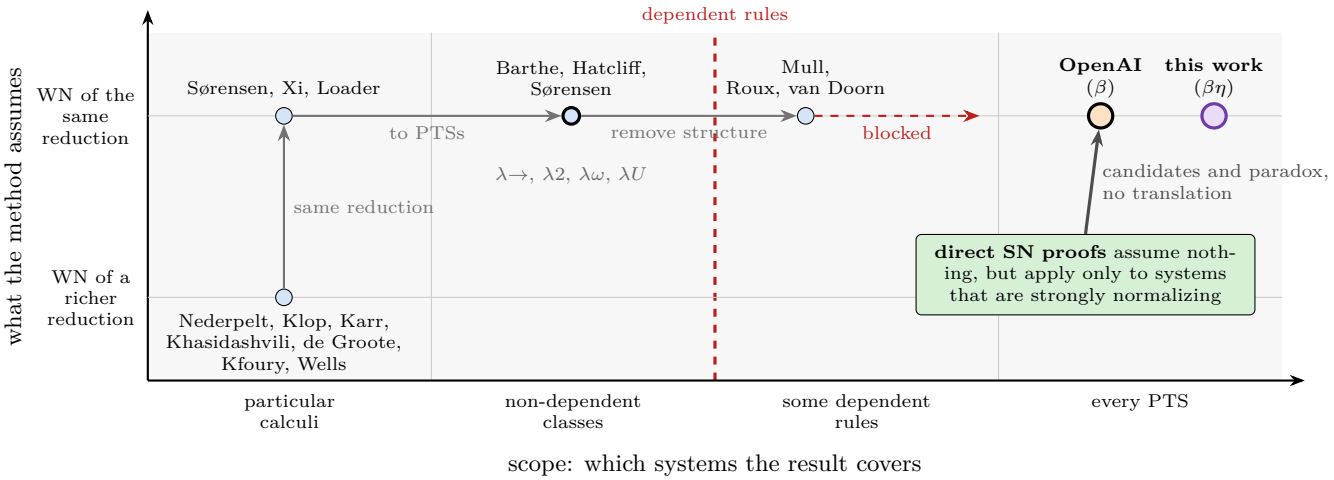


Figure 2: How close each result came to the conjecture, shown qualitatively. The red line marks dependent rules, where every translation result stops, because translating a term first requires translating its type. OpenAI reaches every system from a different direction, starting from the reducibility method of direct proofs. The systems listed under Barthe, Hatcliff and Sørensen are examples from their class.

Two routes. Figure 3 shows why the translation route needs particular rules while the new route does not. A translation must type $[\![M]\!]$ in the same system. A negation requires a rule (s, s, s), and in the presence of dependency, translating a term requires translating its type, which may be larger [39, §3.6]. OpenAI's route builds candidates directly and uses weak normalization only to obtain normal forms and, once, against the paradox.

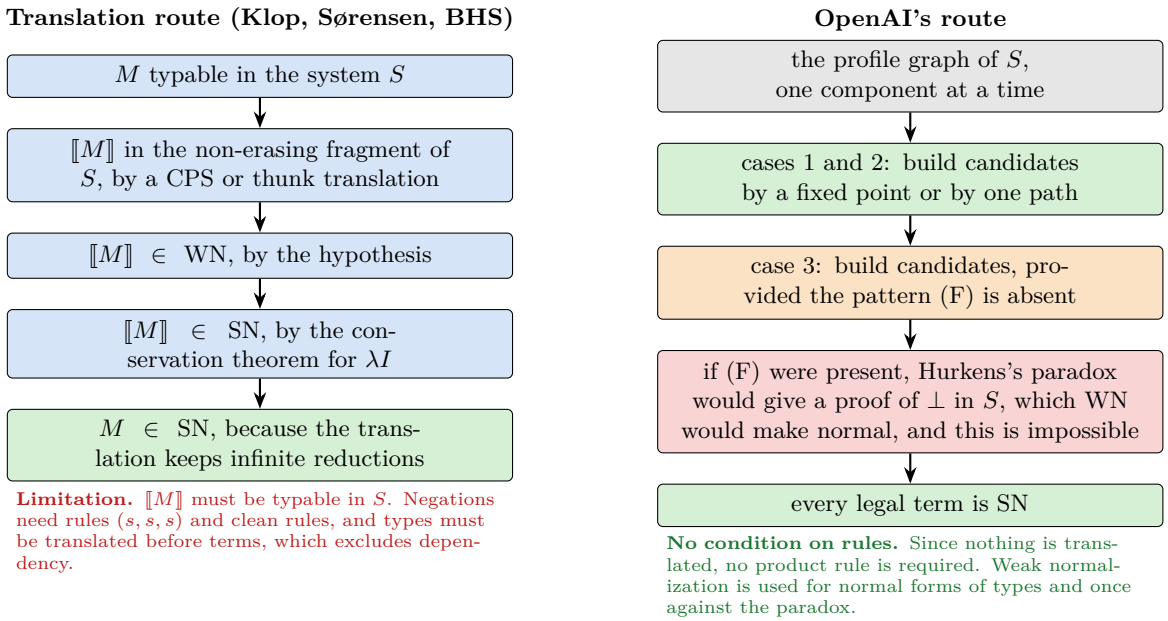


Figure 3: The mechanism of the strongest earlier results (left) and of OpenAI's proof (right). The left route requires suitable product rules inside the system, and the right route requires none.

3 The β proof

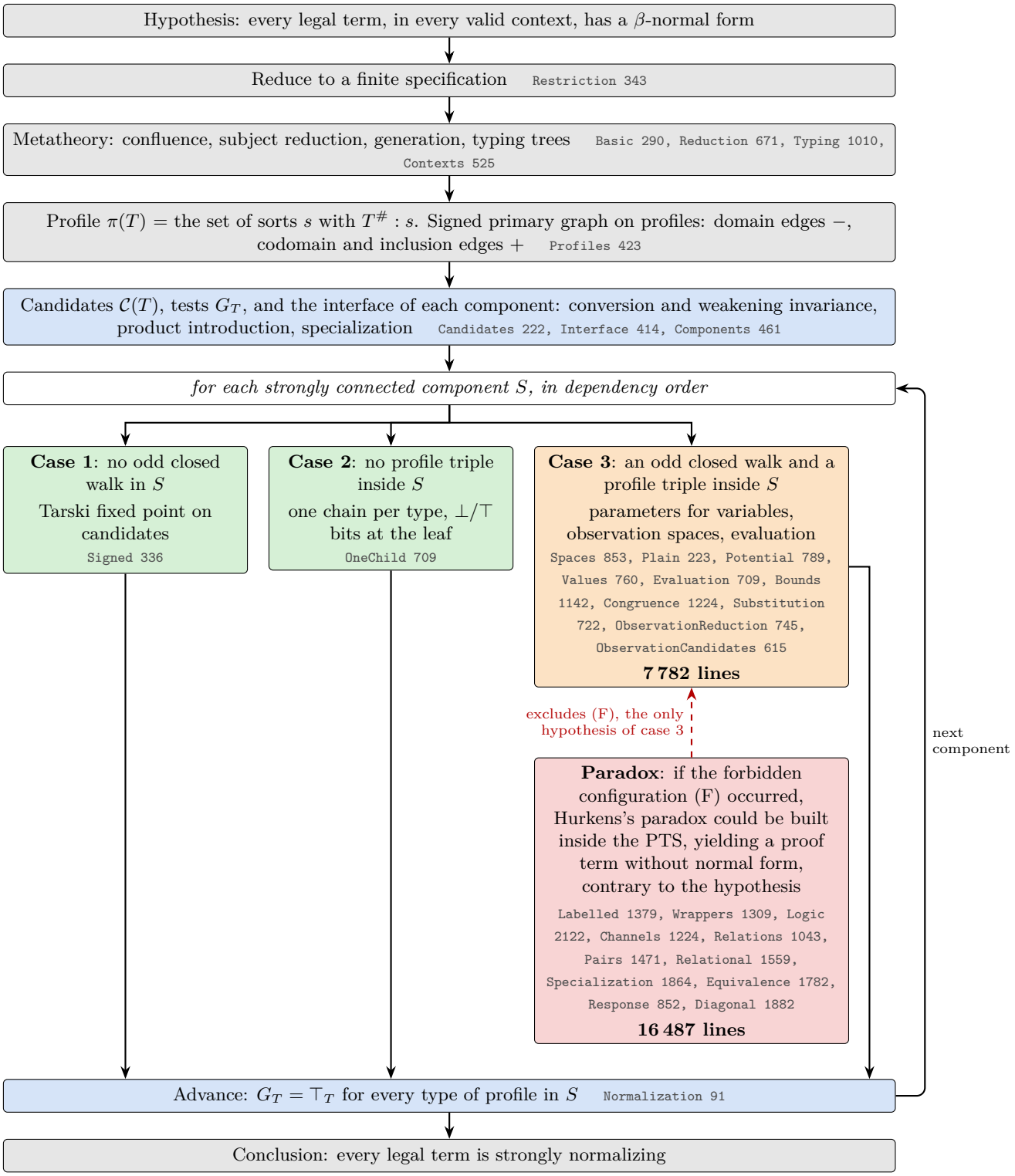
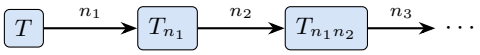


Figure 4: OpenAI's proof and its Lean files, with line counts from the copy in `PTS-beta-eta/PTSBetaEta/Beta` (29 764 lines in total). Gray marks ordinary PTS metatheory, blue the candidate method, green and orange the three constructions, and red the paradox. The paradox makes up 55% of the code and serves only to exclude one finite graph pattern.

4 Candidates and why they are hard to build

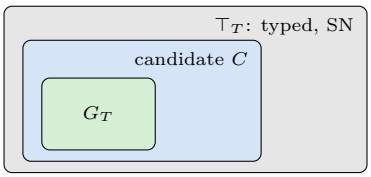
At each actual type T the proof works with three sets of terms (Figure 5). The set $\top_T$ consists of the typed strongly normalizing terms, the set G_T consists of the terms that remain strongly normalizing under every stack of tested arguments, and a candidate is a set lying between the two. The theorem follows once $G_T = \top_T$ is proved for every type.

(a) A tested stack at T



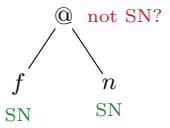
If $T = \Pi x{:}D.E$ and $n_1 \in \top_D$, the next type is $T_{n_1} = (E[x := n_1])^{\#}$, and so on. A term h belongs to G_T when $h\, n_1 \cdots n_k$ is strongly normalizing for every tested stack. The whole proof establishes $G_T = \top_T$ (`TestedStack`, `Ground`).

(b) Candidates lie between G_T and $\top_T$



A candidate must also be closed under head expansion: if $P[x := n]\,\vec{m} \in C$ and the parts are strongly normalizing, then $(\lambda x{:}D.P)\, n\, \vec{m} \in C$. Candidates form a complete lattice with least element $\bot_T$ (`IsCandidate`, `Bottom`). Once $G_T = \top_T$, the candidate is unique.

(c) Why $G_T = \top_T$ everywhere is enough



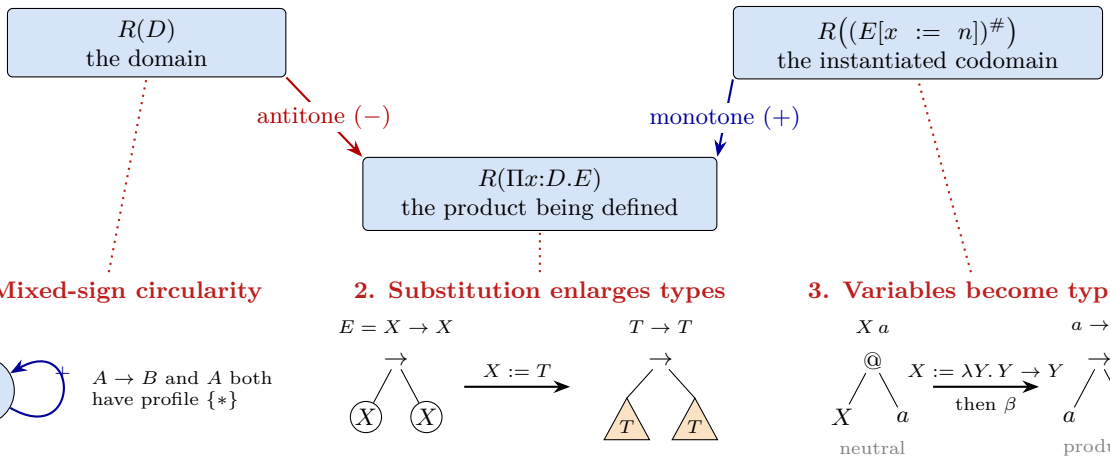
Consider a typed term of least size that is not strongly normalizing. Its children are typed and smaller, hence strongly normalizing. Since sorts, variables, products and λs with strongly normalizing children are strongly normalizing, the term must be an application $f\, n$ with $f : \Pi x{:}D.E$ and $n : D$. Then $f \in \top_T = G_T$ for the normalized product T, and the one-element stack (n) is tested at T, so $f\, n$ is strongly normalizing after all (`all_ground_eq_top`, `application_sn_of_ground`).

Figure 5: The objects of the candidate method. (a) Tests follow the exact successive types. (b) A candidate is any set between the tests and the typed strongly normalizing terms that is closed under head expansion. (c) The invariant $G_T = \top_T$ for every type rules out a smallest counterexample.

Every construction in the proof solves the same equation. For a type $T = \Pi x{:}D.E$ whose profile lies in the current component, the candidate of T must be the set of strongly normalizing terms that map good arguments to good results:

$$R\big(\Pi x{:}D.E\big) \;=\; \big\{\, h \in \mathrm{SN} \;:\; h\, n \in R\big((E[x := n])^{\#}\big) \text{ for every } n \in R(D) \,\big\}.$$

This is Girard's equation for reducibility candidates [21]. In System F it is solved by recursion on the type, but in a general PTS three obstacles prevent this recursion.



With the rule $(*, *)$, a type $A \to B$ and its domain A both have profile $\{*\}$, so R on $\{*\}$ is defined in terms of itself, once antitonely. This gives an odd closed walk, which on its own is harmless because A is a subterm of $A \to B$.

In System F the codomain of $T = \Pi X{:}*\,.\, X \to X$ at the argument T is $T \to T$, which contains T twice, so recursion on types fails. Since profiles only grow, $T \to T$ stays in the component or moves down to an already processed one.

In F_ω a leaf $X\, a$ with $X : * \to *$ is neutral, but after $X := \lambda Y{:}*\,.\, Y \to Y$ it normalizes to the product $a \to a$. The meaning of X must therefore anticipate every type it can become. Girard gives X a parameter, and case 3 does the same.

Figure 6: The equation behind every candidate construction and its three obstacles, shown on familiar systems. Case 1 removes obstacle 1 through sign consistency, and case 2 removes obstacles 2 and 3 through structure. Case 3 faces all three, at the cost of observation spaces and the paradox.

5 The three cases

The case split applies the same test twice, once to candidates and once to the parameter spaces that case 3 introduces. At each level the proof asks two questions about the dependency graph of the definitions it has to make.

1. **Are the signs consistent?** If every closed walk is even, each vertex can be given a sign. The definition then becomes monotone in a mixed order and has an order-theoretic fixed point. For candidates this is Tarski's theorem [44] (case 1), and for parameter spaces it is the alternating chain $C_0 = \emptyset$, $P_r = \mathrm{Tab}(C_r)$, $C_{r+1} = \mathrm{Tab}(P_r)$ (free layers).
2. **Does a measure decrease?** If every dependency points to something smaller, the definition can be made by recursion. For candidates the measure is the syntax of a single chain, which exists when no product has two children in the component (case 2). For parameter spaces it is the count p_C of nodes along the component (plain layers).
3. **Neither.** At the candidate level the proof moves up one level and adds parameters (case 3). At the level of spaces there is no further level to move to. The proof shows that this situation, the *forbidden configuration*, allows the system to encode Hurkens's paradox [23], so it cannot occur in a weakly normalizing system.

This mirrors the familiar trichotomy for recursive type definitions: positive or sign-consistent definitions have solutions, decreasing definitions have solutions, and a definition that is both negative and non-decreasing leads to a Russell-style paradox. The proof turns this third alternative into a contradiction with weak normalization.

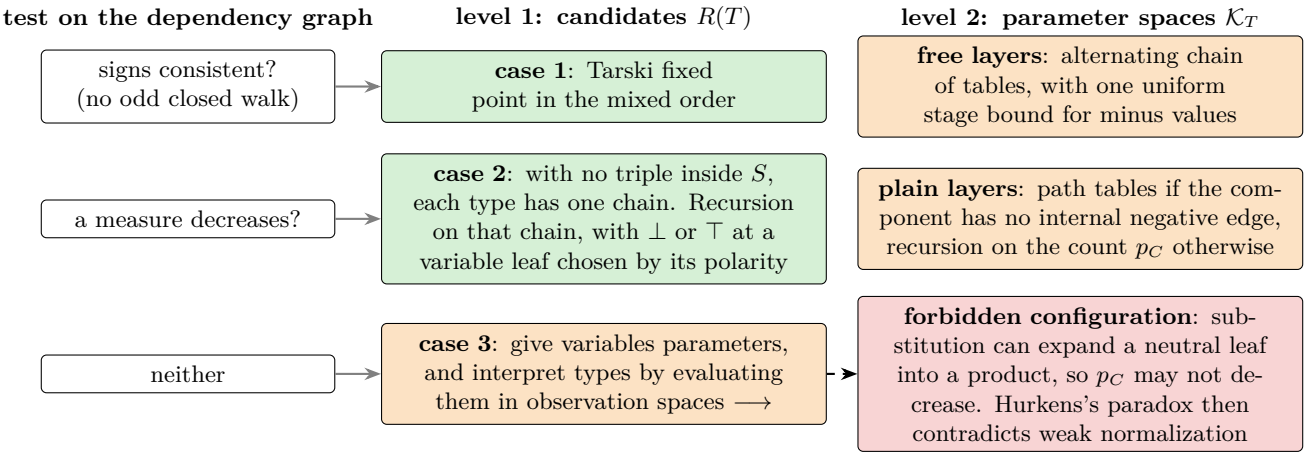
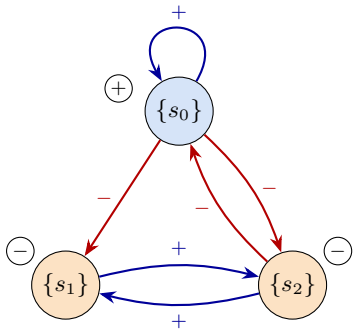


Figure 7: One test applied at two levels. Each row is one way of solving the equation of Figure 6. Only the bottom right cell is impossible, and the paradox shows that it never arises under weak normalization.

The two simple constructions. Figure 8 illustrates cases 1 and 2 on examples. Case 1 needs no syntax at all, since it works on the set of all actual types of the component at once. Case 2 uses syntax, but only one path in each type, and it never looks inside a type produced by substitution.

(a) Case 1 on a component with consistent signs



signs: a + edge keeps the sign, a − edge flips it

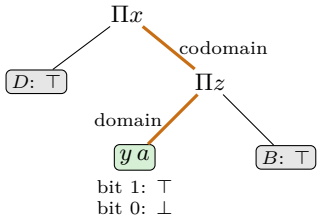
order $\mathcal{C}(T)$ by $\subseteq$ at + types and by $\supseteq$ at − types

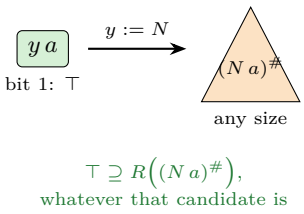
$F(v)_{\Pi x:D.E} = \{h \in \top : \; h\, n \in v_{T_n}$ for all $n \in v_D\}$ is monotone

Tarski: $a = \sup\{v : v \le F(v)\}$ is a fixed point, and $R_T = a_{(T\xi)^\#}$ under a substitution ξ

Because of the negative edge, a domain D inside S has the sign opposite to its product, so the antitone dependence on v_D is monotone in the mixed order. An instantiated codomain inside S has the same sign as the product because of its positive path. Types in processed components are fixed at $\top$. This graph is the global counterexample of Section 7: its only component has (F), and the proof handles it with Tarski's theorem rather than the paradox (`operator`, `step_monotone` in `Signed.lean`).

(b) Case 2: one path inside S, and an extremum at its end



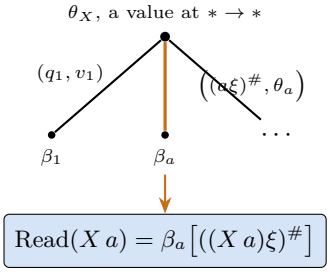


If no product in S has both children in S, every type has a single path inside S. A neutral leaf at its end receives $\bot$ or $\top$, chosen by a bit for its head. Specialization compares candidates before and after a substitution along this path, and each domain step reverses the direction of the comparison. At a leaf headed by the substituted variable, the bit places the extremum on the correct side of every candidate, so the larger substituted type is never inspected (`neutral_compare`, `compare_subst`).

Figure 8: The two simple constructions. (a) Blue vertices have sign + and orange ones sign −, and since every closed walk is even, the signs are well defined. (b) A schematic type $T = \Pi x{:}D.\,\Pi z{:}(y\,a).\,B$, not taken from a particular specification, whose path inside S (orange) goes first to the codomain and then to the domain. Children off the path lie in processed components and receive $\top$. After one domain step the comparison runs backwards, so the leaf needs $\top$.

Case 3. When both tests fail, the proof follows Girard: each variable receives a parameter, and a type is interpreted by evaluating its syntax (Figure 9). The parameter of a variable X of type $\Pi y{:}A.\,s$ must specify which candidate $X\,a$ denotes for every possible argument a, so it is a table indexed by arguments. Specialization, the law for substituting an argument into a codomain, is then proved by transporting a typing tree along substitution and reduction.

(a) A variable carries a value, and a neutral type reads a base



$\mathrm{Val}(x) = \theta_x$

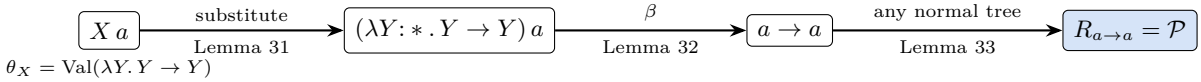
$\mathrm{Val}(F\,N)$ = next value of $\mathrm{Val}(F)$ at the frame $\big((N\xi)^{\#}, \mathrm{Val}\,N\big)$

$\mathrm{Val}(\lambda x.P)$ = the table $(q, v) \;\mapsto\; \mathrm{Val}(P;\, \xi[x := q],\, \theta[x := v])$

$\mathrm{Read}(M{:}s) = \mathrm{base}(\mathrm{Val}\,M)\big[(M\xi)^{\#}\big]$ at a neutral M, and the product test $\mathcal{P}$ at a product

In F_ω, let $X : * \to *$ and $a : *$. The parameter θ_X is a plain value: it has a base and, for every frame (an argument q with a payload v), a next value at $*$. A value at a sort is just a base, that is, an assignment of candidates to all actual types. The candidate of the neutral type $X\,a$ is read off at its own actual expression (`Evaluation.lean`, `preEvaluate`).

(b) Specialization at $X := \lambda Y{:}*\,.\,Y \to Y$

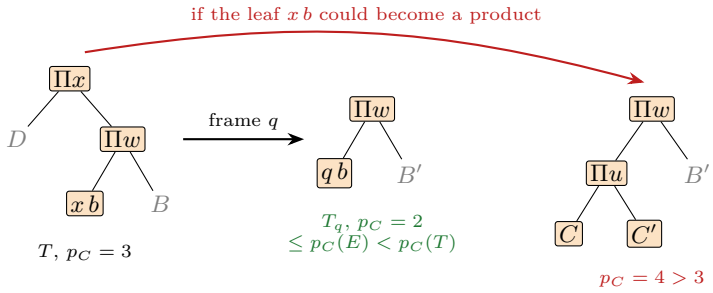


The tree of the normal argument $N^{\#}$ is grafted at X, and its value becomes the parameter of X. Since normal trees are good, substitution preserves the Read (Lemma 31, `Substitution.lean`). Weak normalization reduces the result to its normal form, and reduction preserves good Reads (Lemma 32, `normalize_transport`). The normal form is $(E[X := N])^{\#}$, and all its trees have the same Read (Lemma 33, `normal_tree_independence`). Here the value of $\lambda Y.\,Y \to Y$ at the frame a has base $\mathcal{P}$ at $a \to a$, so the Read of $X\,a$ becomes the product test of $a \to a$. Figure 20 redraws this square for η.

Figure 9: Case 3 on the simplest example. (a) Evaluation assigns values to terms and candidates to types, and the orange frame is the one used by $X\,a$. (b) The specialization law of the interface, proved by three transport lemmas, numbered as in OpenAI's preprint.

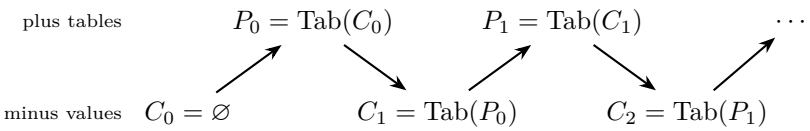
The parameter spaces. The space of θ_X depends on the spaces at the domains and targets of the frames of its type, and these dependencies can form cycles. The proof builds the spaces in two ways (Figure 10), and the forbidden configuration is exactly the case in which neither works.

(a) Plain spaces: recursion on a count p_C of type nodes



$\mathcal{K}_T \simeq \mathcal{B} \times \prod_{\text{frames}} \mathcal{K}_{\text{target}}$, with payloads living at domains. On a plain component with an internal negative edge, the proof recurses on p_C, which counts the nodes that stay in C (orange). A frame target has a smaller count unless substitution expands a neutral leaf $x\,b$ into a product. This requires the leaf to be sorted at some s with $\{s\} \in C$, and the type of x to have a profile I reachable from $\mathrm{Ax}(s)$ by a positive path. Then T yields a triple (I, J, K) with $J, K \in C$, which is exactly (F) (`potential_subst_le`, `frame_potential_drop`).

(b) Free spaces: an alternating chain of tables



A plus table takes minus values as payloads and a minus table takes plus values, and free types can form cycles, so the two spaces are built in alternating stages. The minus space C_* keeps the tables that a single finite stage represents for all their observations, and the plus space is $P_* = \mathrm{Tab}(C_*)$. The finite-bound lemma shows that evaluation stays inside these spaces (`Spaces.lean`, `Bounds.lean`).

Figure 10: The two kinds of parameter spaces in case 3. (a) Orange nodes are counted by p_C and gray ones are not. The left arrow is a legal frame, and the red arrow is the expansion that (F) would permit. The trees are schematic rather than taken from a particular specification, and the count 4 only illustrates that an expansion can raise p_C. (b) A table in $\mathrm{Tab}(Y)$ gives a base observation for every continuing stack of frames and an exit observation for every exiting stack, and the opposite-sign payloads in these stacks come from Y.

6 The forbidden configuration on familiar systems

The forbidden configuration (F) is the configuration prohibited by condition (7) in Section 5 of OpenAI's preprint [35]. It involves the current component S and a strongly connected piece C of the *secondary graph*. The vertices of the secondary graph are the profiles reached by positive paths from the layers $\mathrm{Ax}(s)$ with $\{s\} \in S$. Its edges are the positive edges between these vertices, together with the negative edge $I \to K$ of each triple (I, J, K) whose three profiles are all vertices. A vertex is *direct* if it is the domain I of a triple (I, J, K) with $J, K \in S$. The configuration requires the following.

(a) C is plain (it reaches direct vertices along paths of both parities) and has an internal negative edge.

(b) There is a profile triple (I, J, K) with $J, K \in C$.

(c) There is a sort s with $\{s\} \in C$ and $\mathrm{Ax}(s) \neq \emptyset$, so that C contains *variables* of sort s.

(d) There is a positive path $\mathrm{Ax}(s) \to I$, so that a product in C can *quantify over* the inhabitants of s.

Informally, (F) is a layer of types that has variables and can quantify over itself. Among the systems of Table 1, the configuration occurs exactly in λU^-, λU and $\lambda *$, the three systems that are not normalizing. The sharpest contrast is between λHOL and λU^-, which differ only by the rule $(\triangle, \square)$. This rule creates the edge $\{\triangle\} \to \{\square\}$ that completes the configuration.

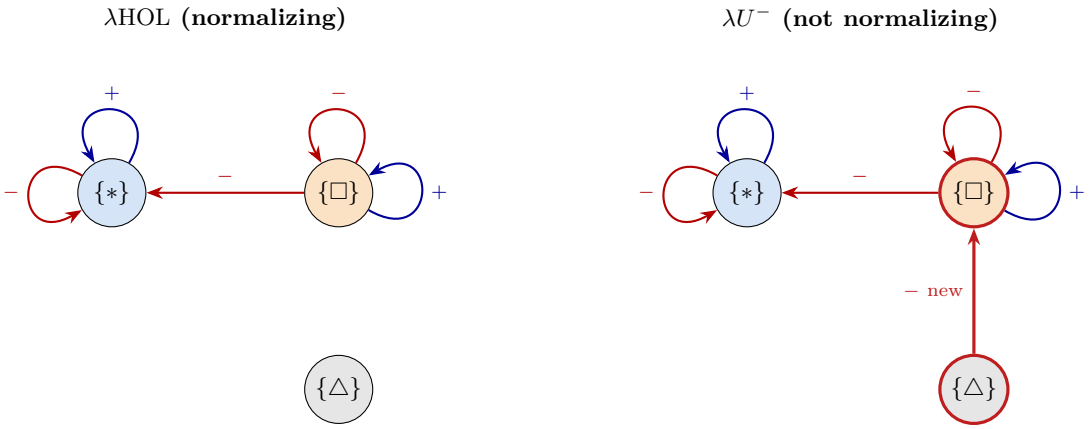


Figure 11: Primary graphs of λHOL and λU^-. Red arrows are domain edges ($-$) and blue arrows codomain edges ($+$). For the component $S = \{\{*\}\}$, the secondary graph consists of the single vertex $\{\square\} = \mathrm{Ax}(*)$ with its negative loop, so in both systems $\{\square\}$ is plain with an internal negative edge. Conditions (c) and (d) hold only in λU^-.

system	components and constructions used	paradox needed
$\lambda\!\to$	$\{\square\}$ case 1, $\{*\}$ case 3 without parameters	no
$\lambda 2$ (System F)	$\{\square\}$ case 1, $\{*\}$ case 3 with a free plus layer $\{\square\}$	no
λP (LF)	$\{\square\}$ case 1, $\{*\}$ case 3 without parameters	no
$\lambda\omega$ (F_ω)	$\{\square\}$ case 3 without parameters, $\{*\}$ case 3 with a plain layer (count)	no
λC (CC)	$\{*, \square\}$ case 3 with a plain layer (count)	no
λHOL	$\{\triangle\}$ case 1, $\{\square\}$ case 3, $\{*\}$ case 3 with a plain layer (count)	no
λU^-, λU	as λHOL, plus a free plus layer, and (F) at $\{*\}$	yes
$\lambda *$	$\{*\}$ case 3, plain layer, (F) at $\{*\}$	yes

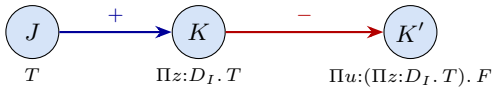
Table 1: Components of the standard systems and the constructions that handle them, computed from the definitions in `Profiles.lean`. A free or plain layer is a vertex of the secondary graph whose parameter space is built by case 3. No system in the table uses case 2 or a free minus layer, so none of them exercises the uniform stage bounds of `Bounds.lean`.

How the paradox excludes (F). Figure 12 presents the paradox half as a sequence of steps. Steps 1 to 6 build, inside the given PTS, a proof of a terminal formula from the forbidden configuration, and steps 7 and 8 turn weak normalization into a contradiction. Only step 7 uses the hypothesis.

① **Labelled calculus.** Every application, λ and product carries a label dd, dp or pp, giving the modes (data d or proof p) of its domain and its result. Variables receive modes from the context. Erasure $|\cdot|$ forgets the labels but keeps every annotation, and every β-step of an erased typed term lifts to a labelled step. `Labelled 1379`

↓

② **Wrappers.** A path in the profile graph wraps a type one edge at a time (inset). A positive edge adds a dummy domain, which a projection removes up to β. A negative edge sends T to $\Pi z{:}T.\,F$ and thereby reverses the direction in which proofs flow. `Wrappers 1309`



The + edge of a triple (I, J, K) adds a dummy domain, with $\mathrm{pr}(\mathrm{up}\, t) = t$ up to β. The − edge of a triple (K, J', K') uses a fixed type F sorted at J'.

↓

③ **Logic.** Formulas are stable types sorted at the logic vertex b. The all-S triple (a_0, b, k_0) provides implication, a direct vertex provides $\forall$ over its data, and $\bot$ is the terminal F_b. The logic is classical. Data wrappers store a formula as data of a sort u with $\{u\} \in S$, with $\mathrm{decode}(\mathrm{encode}\, P) \leftrightarrow P$. `Logic 2122`

↓

④ **Data channels.** Predicates on a data type satisfy $\mathrm{pred}(x.\Phi)@w \;\leftrightarrow\; \Phi[x := \mathrm{Round}\, w]$. A bundle keeps one data parameter exact, yielding a probe type that does not depend on that choice. All of this holds only up to logical equivalence, never as an equality inside data. `Channels 1224`

↓

⑤ **Relational coding.** Predicates on data are compared up to observational equivalence. Three tagged copies and one admissible specialization yield the class and pairing laws needed by the diagonal.
`Relations 1043, Pairs 1471, Relational 1559, Specialization 1864, Equivalence 1782, Response 852`

↓

⑥ **Guarded diagonal.** Hurkens's paradox in Geuvers's presentation [23, 15], with $Q(x) = \forall i.\,\mathrm{Ext}(i) \Rightarrow (\mathrm{le}(i, x) \Rightarrow i@\delta x)$ and $j_* = \mathrm{pred}(\neg Q)$, yields a proof $\Delta \vdash P : F_b$. `Diagonal 1882`

↓

⑦ **Erase and normalize.** $|P|$ is legal in the original PTS. Weak normalization reduces it to a normal form, and step 1 lifts this reduction to a proof $P' : F_b$ that is still in proof mode and has normal erasure.

↓

⑧ **No normal proof at a terminal.** A smallest such P' is either a λ whose body is a smaller such proof, or a spine headed by a converter c whose first argument is a smaller such proof. Hence (F) cannot occur. `no_proof_at_target`

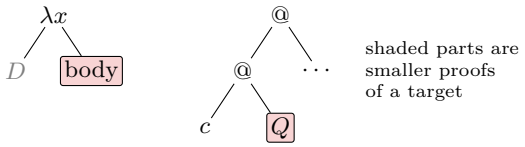


Figure 12: The paradox half, step by step. Steps 1 to 6 build a proof of the terminal formula F_b from the forbidden configuration in a single finite labelled context, and steps 7 and 8 turn weak normalization into a contradiction. In Lean the chain consists of `LogicSetup.exists_setup`, `ChannelSetup.exists_setup_return`, `relational_bottom` and `no_proof_at_target`, assembled in `relational_exclusion`.

7 A shorter proof?

The candidate half consists of three constructions, and the paradox accounts for more than half of OpenAI's Lean code (Figure 13). This section asks which parts are necessary and where a shorter proof might come from. Every notion used here can be computed on finite graphs from the definitions in `Profiles.lean`: feasible profiles, the primary graph, components, the secondary graph, plain and free layers, and the forbidden configuration. The two counterexamples below can be verified by hand from these definitions, and the frequencies are counts over random specifications with two to five sorts.

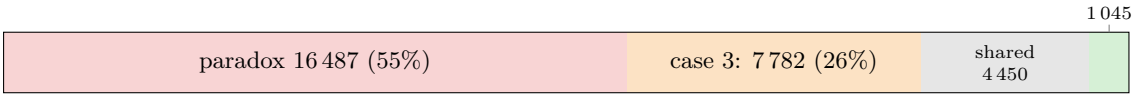
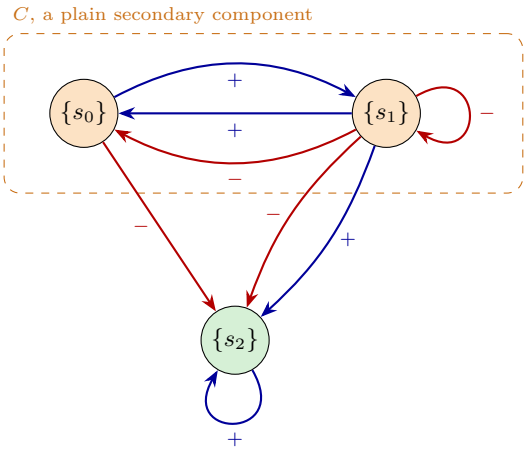


Figure 13: Lean lines of OpenAI's proof, by part. Cases 1 and 2 form the thin green slice on the right.

Cases 1 and 2 cannot be merged into case 3. Case 3 needs only the absence of the forbidden configuration (F) at the current component. If (F) at a component always forced an odd closed walk and an internal triple there, the paradox would exclude (F) everywhere and cases 1 and 2 could be deleted. Two specifications show that this fails, in two forms.

- *Local form.* Take sorts s_0, s_1, s_2 with axioms $s_1{:}s_1$, $s_2{:}s_1$ and rules (s_0, s_1, s_2), (s_1, s_0, s_1), (s_1, s_1, s_0), (s_1, s_2, s_2). The component $\{\{s_2\}\}$ has no odd closed walk but has (F).
- *Global form*, in which (F) at some case-1 or case-2 component would at least imply (F) at some case-3 component. Take axioms $s_1{:}s_1$, $s_2{:}s_0$ and rules (s_0, s_1, s_2), (s_0, s_2, s_1), (s_2, s_0, s_0). The only component $\{\{s_0\}, \{s_1\}, \{s_2\}\}$ has no odd closed walk but has (F). There is no case-3 component to host the paradox, and the proof handles this system with Tarski's theorem alone. Figure 8(a) shows this graph.



Axioms $s_1{:}s_1$ and $s_2{:}s_1$, rules (s_0, s_1, s_2), (s_1, s_0, s_1), (s_1, s_1, s_0), (s_1, s_2, s_2). The component $\{\{s_2\}\}$ (green) has only a positive loop, so case 1 handles it, yet it has (F). Its secondary graph starts at $\mathrm{Ax}(s_2) = \{s_1\}$, the triple $(\{s_1\}, \{s_2\}, \{s_2\})$ makes $\{s_1\}$ direct, and $C = \{\{s_0\}, \{s_1\}\}$ is plain, with the negative loop at $\{s_1\}$. The sort s_1 gives $\{s_1\} \in C$ with $\mathrm{Ax}(s_1) = \{s_1\}$, which reaches $I = \{s_1\}$ by the empty path. The other component $\{\{s_0\}, \{s_1\}\}$ requires case 3 and also has (F), so the paradox still excludes (F) somewhere in this system.

Figure 14: The local counterexample: a component can have (F) without an odd closed walk. In this system a case-3 component has (F) as well, whereas the global counterexample of Figure 8(a) has no such component.

The three constructions therefore complement each other. Since they make up only 3.5% of the code, this matters more for understanding the proof than for its length.

Free minus layers are rare. Free minus layers occur in none of the standard systems (Table 1), in 1 of 20 000 random 3-sort specifications, and in 6 of 28 000 random functional specifications with 4 or 5 sorts. They therefore do not require non-functional rules, only unusual ones. Without minus layers a plus table has no opposite-sign payloads, so the alternating chain is unnecessary, and we expect the same of the uniform bounds.

Many forbidden configurations are not shaped like λU^-. The paradox needs a way to quantify over a layer of types inside that same layer, together with a logic in S. Rocq's `Logic/Hurkens.v` [46] proves the paradox for an abstract system with two universes in its module `Generic`, in about 180 lines with a 60-line core. It needs no reduction rule at all for the small (formula) universe, but exact β for three things at the large (data) universe: products over data that return to the data sort, quantification over all data types that returns to it, and a sort of formulas inside the data universe. We call a configuration *tight* if it contains such a core, with data-level products that return to the

data sort along positive edges, where OpenAI's wrappers cancel up to β. Among random specifications that need the paradox, the counts are as follows.

- Tight cores exist in about 60% of these specifications with 2 sorts and in about 40% of those with 3 or 4 sorts. In the remaining ones at least one of the three data-level requirements fails, and the failures are spread over all three requirements with no dominant one.
- The return path from the quantified product K to the data layer $\{s\}$ is positive in 99.6% of the configurations (6 959 of 6 989), so OpenAI's wrappers would cancel up to β there. This does not hold in general: the axioms $s_0{:}s_0$, $s_0{:}s_1$, $s_0{:}s_2$, $s_1{:}s_0$ with the rules (s_0, s_2, s_2), (s_1, s_2, s_2), (s_2, s_0, s_2), (s_2, s_1, s_0), (s_2, s_2, s_2) give (F) but no positive return path.

OpenAI's general encoding, with its relational coding up to logical equivalence, is therefore needed. The generic paradox could serve as a fast path for about half of the configurations, but it would add a second route rather than remove the relational layer. The open question for a substantially shorter paradox is whether predicates over data, which OpenAI builds as wrappers into terminal types, can be given exact β in every configuration. Since such a statement is a lemma about finite graphs, it can be tested on many specifications before any work in Lean. Table 2 summarizes the section.

part	assessment
cases 1 and 2	Needed: the two counterexamples above show that case 3 cannot take them over.
free minus layers	Needed in general, but never by the standard systems. A first version of the proof for specifications without minus layers could skip the uniform bounds (most of `Bounds.lean`, 1 142 lines, and the minus half of `Spaces.lean`). Whether a finite specification has free minus layers is decided by a finite graph computation.
paradox, generic part	The relational coding (`Relations`, `Pairs`, `Relational`, `Specialization`, `Equivalence`, `Response`, about 8 600 lines) exists because data-level β holds only up to $\leftrightarrow$ along negative wrappers. For tight configurations, Rocq's generic Hurkens paradox (about 180 lines) should apply with exact β. This covers roughly half of the random cases, so it would be a second route rather than a replacement.
paradox, encoding part	`Labelled`, `Wrappers`, `Logic` and `Channels` (about 6 000 lines) build a logic out of an arbitrary profile graph. A smarter choice of paths could shorten this part, for instance by proving that every configuration contains a smaller one with fewer negative edges on its return paths.

Table 2: Assessment of each part. The first row is settled by the two counterexamples. The second rests on counts over specifications, and its saving is expected but has not been checked in Lean. The last two rows are directions, not results.

8 The $\beta\eta$ proof

The development `PTS-beta-eta` [36] proves the same implication for Geuvers's systems $\mathrm{PTS}_{\beta\eta}$ [18], whose conversion rule uses $\beta\eta$-conversion: if every legal term is weakly $\beta\eta$- or weakly β-normalizing, then every legal term is strongly $\beta\eta$-normalizing. OpenAI's theorem does not imply this, because $\mathrm{PTS}_{\beta\eta}$ can type more terms than PTS_{β}, so the proof reruns OpenAI's argument inside $\mathrm{PTS}_{\beta\eta}$ (Figure 15). The recursion over components is unchanged, and the new material comes before it, after it and inside case 3. Notation is explained below the figure.

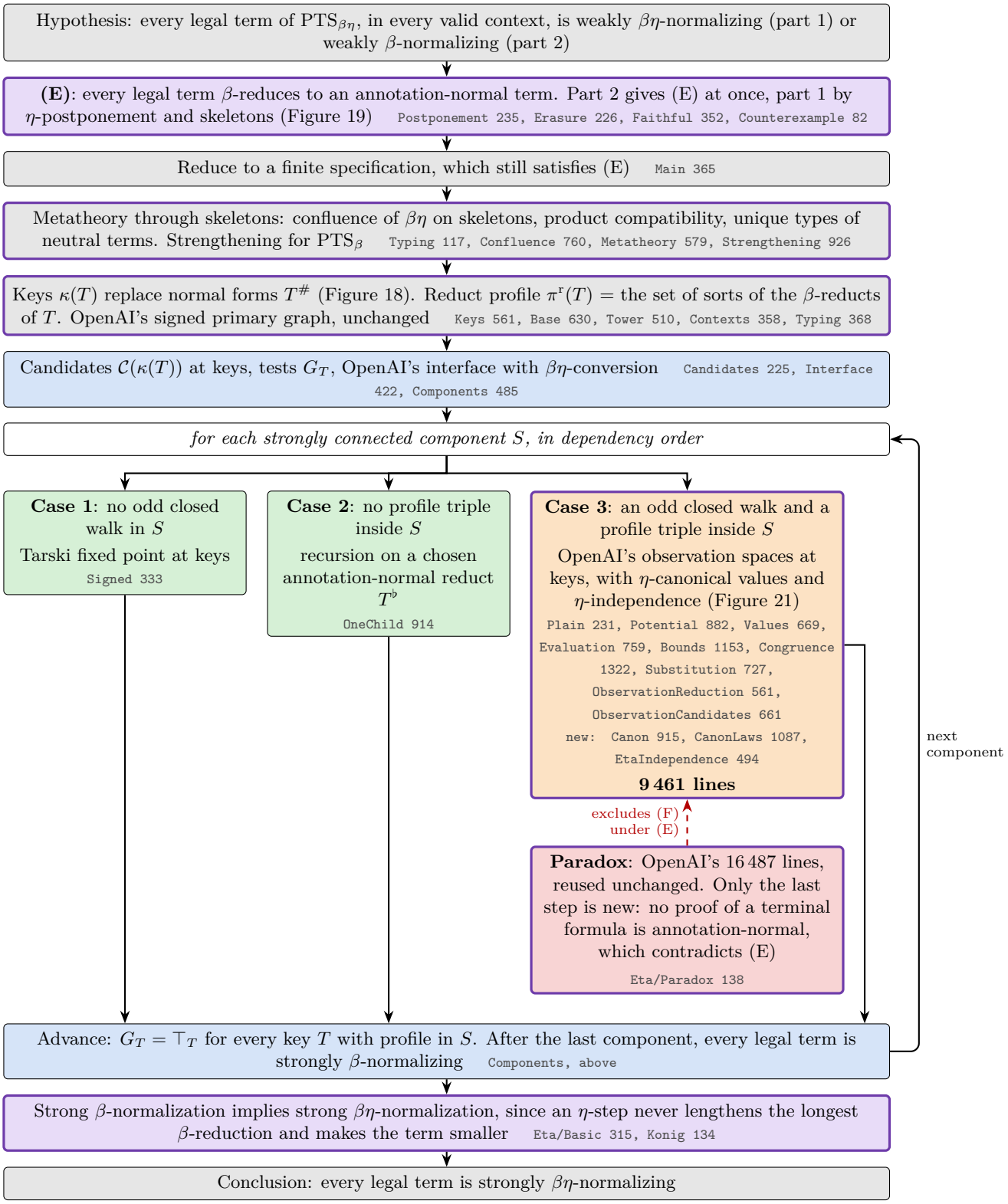


Figure 15: The $\beta\eta$ proof and its Lean files in `PTS-beta-eta`, 48 260 lines in total (`Beta` 29 764, `Normalization` 14 267, `Eta` 3 864, `Main` 365). Fill colors are as in Figure 4. Violet boxes are steps absent from the β proof, and a violet frame marks a step whose mathematics changes. The remaining steps are OpenAI's, with keys in place of normal forms.

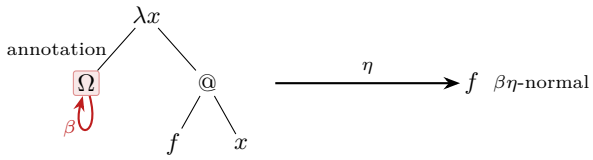
Notation. Three pieces of notation are used from now on.

- The *skeleton* M° replaces every λ-annotation of M by a fixed sort $\star$ but keeps product domains.
- The *key* $\kappa(M)$ is the $\beta\eta$-normal form of M° when that normal form exists (`KWN M` in Lean). Under (E), this holds for every legal term. Lean's total function `knf` returns M° otherwise.
- A term is *annotation-normal* if all its β-redexes lie inside λ-annotations (`ENormal`, for "erasure-normal", in Lean), and $M^\flat$ denotes a chosen annotation-normal β-reduct of M.

9 What η changes

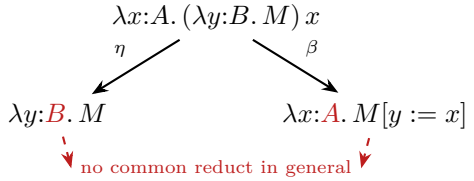
Figure 16 shows the four places where the β proof relies on something that fails in the presence of η, and Figure 17 lists the replacement for each. The remaining figures present the replacements one at a time.

1. η deletes annotations



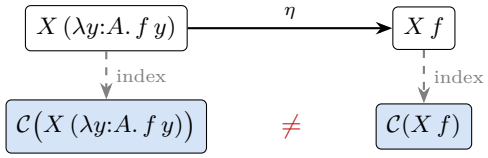
The pseudo-term $\lambda x{:}\Omega.\, f\, x$ η-reduces to the $\beta\eta$-normal term f. Its only β-redex is $\Omega = (\lambda y{:}D.\, y\, y)\,(\lambda y{:}D.\, y\, y)$, which sits in the annotation and reduces only to itself. Thus weak $\beta\eta$-normalization of an individual pseudo-term need not give a β-normal form.

2. $\beta\eta$ is not confluent



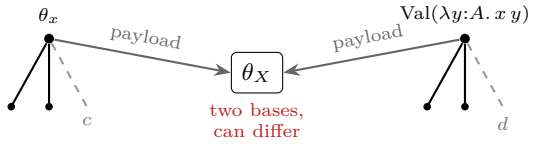
Nederpelt's peak on annotated terms [34]: the two reducts carry different annotations. OpenAI derives product compatibility and uniqueness of the types of neutral terms from confluence of β, and both facts are used throughout. With η they require a different proof.

3. Convertible β-normal types differ



Both types are β-normal and $\beta\eta$-convertible, yet they are different terms. Indexing by β-normal forms gives them two indices, although the conversion rule moves terms freely between the two types.

4. Observation values see η



Let $x : \Pi y{:}A.B$. Its value θ_x and the value of $\lambda y{:}A.\, x\, y$ agree on every legal frame (black) and differ only in data that no observation reads (gray). A parameter θ_X of $X : (\Pi y{:}A.B) \to s$ receives either value as payload and can react to the gray part, so $X\, x$ and $X\,(\lambda y{:}A.\, x\, y)$ can receive different candidates.

Figure 16: The four places where the β proof relies on something that fails with η. Points 1 to 3 concern syntax and are repaired by the hypothesis (E), skeletons and keys (Figures 18 and 19). Point 4 concerns the semantic construction of case 3 and requires a new idea (Figure 21).

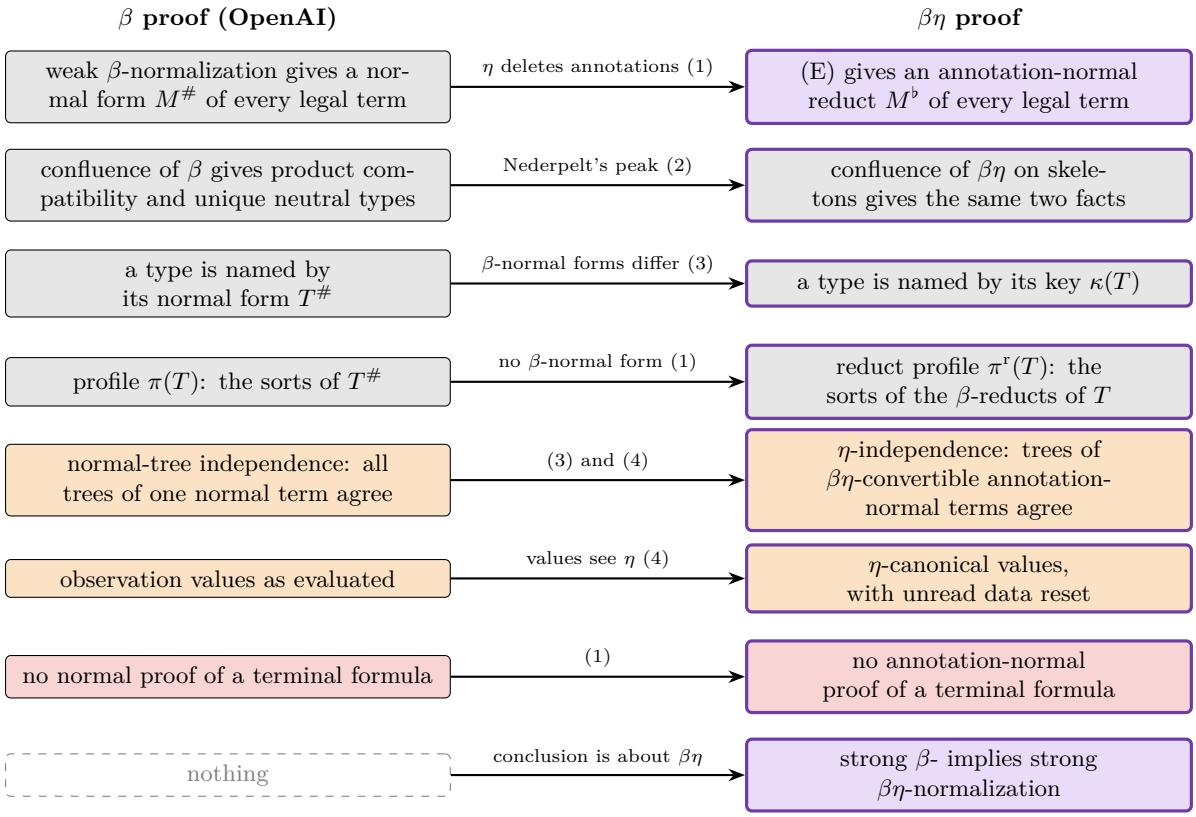
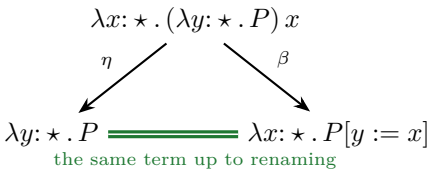


Figure 17: What the $\beta\eta$ proof replaces. Each row pairs an ingredient of the β proof with its replacement, and the arrow names the point of Figure 16 that forces the change. Every other part of the β proof is reused as it stands.

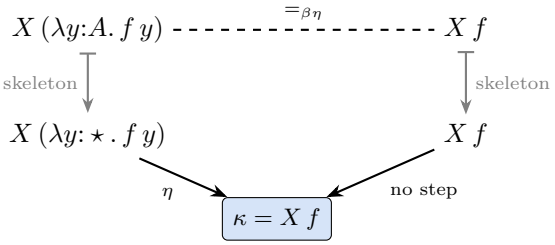
Skeletons and keys. OpenAI uses the normal form of a type as its name. With η, however, convertible β-normal types can differ, and weak $\beta\eta$-normalization of an individual pseudo-term need not give a β-normal form. The $\beta\eta$ proof therefore names a type by its key. Keys are computed from skeletons, on which $\beta\eta$ is confluent again (Figure 18).

Nederpelt's peak closes on skeletons



In a skeleton every annotation is $\star$, so the two sides of the peak in Figure 16 coincide and $\beta\eta$ is confluent on skeletons (`Eta/Confluence.lean`). This yields product compatibility on pseudo-terms. Convertible neutral terms typed in the same context have convertible types, without any functionality assumption.

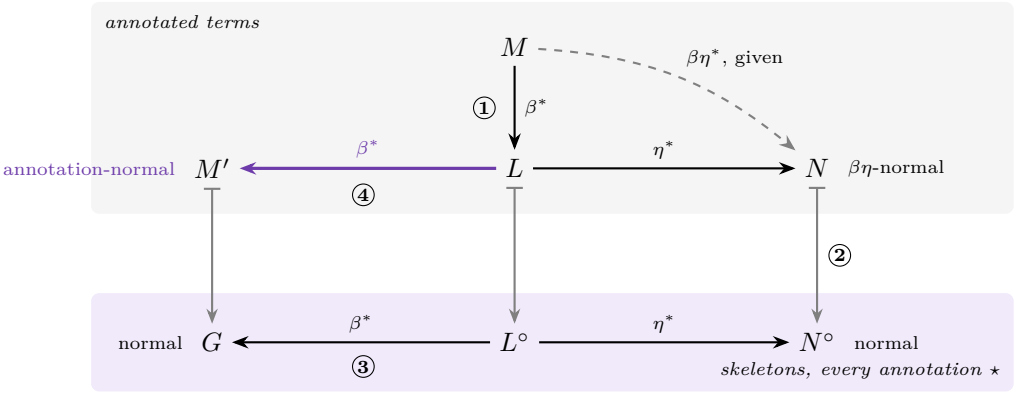
Two convertible types get one key



The domain lemma gives $M =_{\beta\eta} M^\circ$. If M° has a $\beta\eta$-normal form, it is the key $\kappa(M)$, unique by confluence on skeletons. Two terms with such normal forms have the same key exactly when $M =_{\beta\eta} N$, again without any typing hypothesis. Under (E) every legal term has a key, namely the η-normal form of $(M^\flat)^\circ$.

Figure 18: Skeletons repair point 2 and keys repair point 3 of Figure 16. In the example of point 3 both types have key $X\ f$. Product domains are kept in the skeleton, because typing inspects them and η never deletes them.

The hypothesis (E). OpenAI's argument normalizes legal terms in a few places: actual types, argument keys, substituted types and the paradox. With η the construction uses annotation-normal reducts (Figure 19). Each of these uses was checked to work with such reducts, for the same reason every time: the argument inspects λ-bodies and application spines, but never λ-annotations.



① η-postponement [43] moves all η-steps of the given sequence to its end. ② Erasing annotations either keeps an η-step or makes it trivial, and N° is normal because N is. ③ A skeleton that η-reduces to a normal term is weakly β-normalizing. Here the skeleton matters: since all annotations are $\star$, Nederpelt's peak cannot occur. ④ Every β-step of a skeleton comes from a β-step of the term, so $L^\circ \to_\beta^* G$ lifts to $L \to_\beta^* M'$ with $M'^\circ = G$. A term with a normal skeleton is annotation-normal.

Figure 19: From weak $\beta\eta$-normalization to (E). This step requires no normality of the annotations of M'. For arbitrary pseudo-terms they may diverge, as in $\lambda x{:}\Omega.\, f\, x$.

Substitution. Substitution is where the β proof depends most on normal forms. The codomain $E[x := n]$ of a product is not a subterm of the product, so OpenAI normalizes it again, and uniqueness of normal forms makes the result canonical. Figure 20 shows what happens with η. Panel (a) shows that substitution produces η-variants of the same type. Panel (b) depicts the specialization law of the interface, which substitutes an argument into a codomain, as a square that closes in the β proof but only up to $\beta\eta$ in the $\beta\eta$ proof.

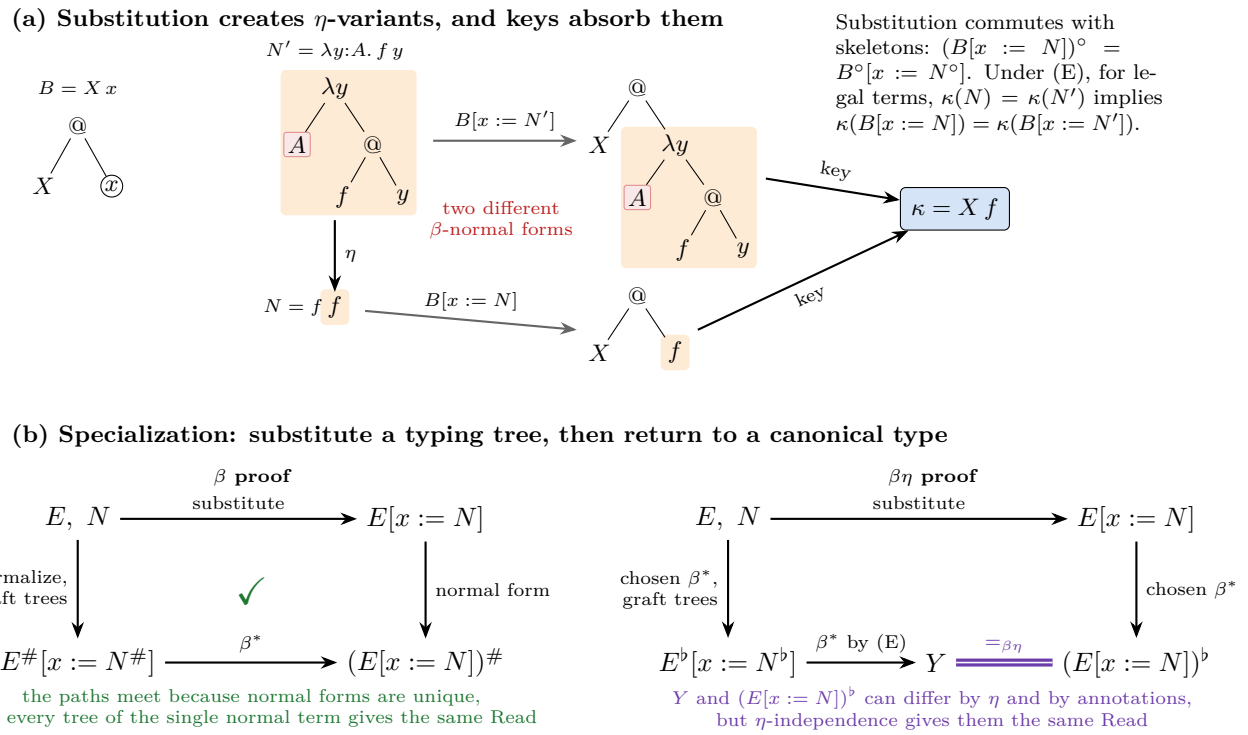
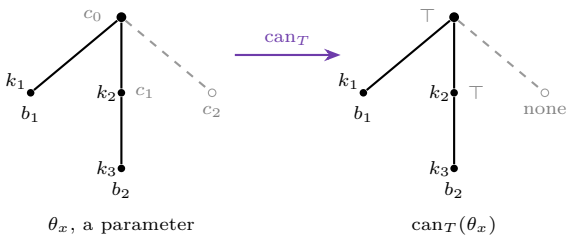


Figure 20: Substitution with and without η. (a) Arguments that are equal up to η yield two different β-normal types, $X\,(\lambda y{:}A.\, f\, y)$ and $X\, f$, which the β proof would treat as two indices, although their keys are equal. (b) The tree of the argument is grafted into the tree of the codomain at x, with the value of the argument as payload. In the $\beta\eta$ proof this value is canonical, so the new variable clause reads it unchanged. The square closes exactly in the β proof, and in the $\beta\eta$ proof it closes only up to $\beta\eta$, by η-independence (Figure 21).

η-canonical values. Point 4 is the only substantive change. A value in case 3 is a table indexed by frames, and evaluation reads only part of this table. The proof defines a canonical form $\mathrm{can}_T(w)$ that keeps the part that is read and resets the rest (`canonValue`), and a variable now evaluates to the canonical form of its parameter. Two laws then make η hold exactly for values: application commutes with can, and $\mathrm{Lam}\big((k,p) \mapsto \mathrm{can}(\mathrm{App}(w,k,\mathrm{can}\ p))\big) = \mathrm{can}(w)$ whenever T is not a sort (`lambdaValue_eta`). Moreover, every evaluated value is canonical (`preEvaluate_canonical`). Figure 21 shows the definition and the one new case of the invariance lemma.

A value at $T = \Pi y{:}A.B$, and its canonical form

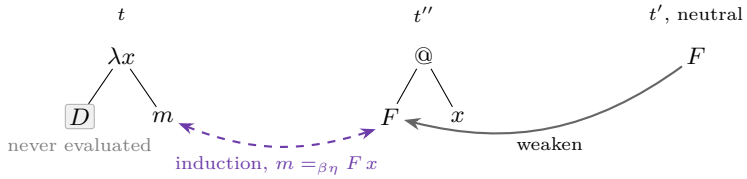


$$\mathrm{Val}(x) = \mathrm{can}_T(\theta_x) \qquad \text{new variable clause}$$

$$\mathrm{Val}(\lambda y{:}A.\, x\, y) = \mathrm{Lam}\big((k,p) \mapsto \mathrm{App}(\mathrm{can}_T\, \theta_x,\, k,\, \mathrm{can}\ p)\big)$$
$$= \mathrm{can}_T(\theta_x) \qquad \eta\text{-law for values}$$

Hence x and $\lambda y{:}A.\, x\, y$ get the same value, and point 4 of Figure 16 disappears. The black entries b_1, b_2 are bases at sorts reached after legal frames. Gray entries are never read: the base at a type that is not a sort, everything below an illegal frame (dashed), and the payloads that carry such data.

The new case of the invariance lemma: an abstraction against a neutral term



The size satisfies $|\lambda x{:}A.\, M| = |M| + 3$ and ignores annotations (`etaSize`). Hence $|m| + |F\, x| < |\lambda x{:}D.\, m| + |F|$, and the induction hypothesis applies to the body m and the tree t'' of $F\, x$.

$$\mathrm{Val}(t) \overset{\text{induction}}{=} \mathrm{Lam}\big((k,p) \mapsto \mathrm{App}(w,k,\mathrm{can}\ p)\big) \overset{\eta\text{-law}}{=} \mathrm{can}(w) \overset{w \text{ canonical}}{=} w = \mathrm{Val}(t')$$

Figure 21: η-canonical values. Top: canonicalization keeps what observations read and resets the rest, and with the new variable clause x and $\lambda y{:}A.\, x\, y$ receive the same value. Bottom: the η-case of the lemma that replaces OpenAI's normal-tree independence (`eta_case` in `EtaIndependence.lean`), where $w = \mathrm{Val}(t')$. Since annotations are never evaluated, the cases of two abstractions and of two neutral terms go through as in OpenAI's proof.

What stays the same. The recursion over components, the three cases, the profile graph and the paradox are OpenAI's. Reduct profiles of $\mathrm{PTS}_{\beta\eta}$ are feasible profiles of OpenAI's graph, so the same graph decides the case split. In particular, Table 1 and the counterexamples and counts of Section 7 apply to $\mathrm{PTS}_{\beta\eta}$ without change. The $\beta\eta$ proof is not shorter than the β proof (Figure 22). Rather, it is a complete check that OpenAI's argument survives the weaker hypothesis (E), together with one new idea, the η-canonical values.

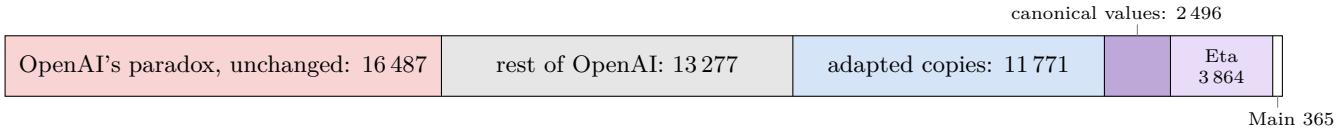


Figure 22: Lean lines of `PTS-beta-eta` by origin. The adapted copies are OpenAI's candidate half, rewritten for keys and annotation-normal reducts. The two violet slices are the η-specific files: `Canon`, `CanonLaws` and `EtaIndependence`, and the directory `Eta`.

Acknowledgements

The figures follow the argument and Lean formalization of OpenAI [35] and the Lean development `PTS-beta-eta` [36], and cite the corresponding Lean files throughout. This guide, its figures and the counts in Section 7 were prepared with the help of Claude Opus 5.5, used through Claude Code.

References

[1] Henk P. Barendregt. Lambda calculi with types. In S. Abramsky, D. M. Gabbay, and T. S. E. Maibaum, editors, *Handbook of Logic in Computer Science*, volume 2, pages 117–309. Oxford University Press, 1992.

[2] Henk P. Barendregt, Jan A. Bergstra, Jan Willem Klop, and Henri Volken. Degrees, reductions and representability in the lambda calculus. Preprint 22, Department of Mathematics, University of Utrecht, 1976.

[3] Gilles Barthe. Existence and uniqueness of normal forms in pure type systems with $\beta\eta$-conversion. In *Computer Science Logic (CSL '98)*, volume 1584 of *Lecture Notes in Computer Science*, pages 241–259. Springer, 1999.

[4] Gilles Barthe and Thierry Coquand. Remarks on the equational theory of non-normalizing pure type systems. *Journal of Functional Programming*, 16(2):137–155, 2006.

[5] Gilles Barthe, John Hatcliff, and Morten Heine Sørensen. CPS translations and applications: the cube and beyond. *Higher-Order and Symbolic Computation*, 12(2):125–170, 1999.

[6] Gilles Barthe, John Hatcliff, and Morten Heine Sørensen. Weak normalization implies strong normalization in a class of non-dependent pure type systems. *Theoretical Computer Science*, 269(1–2):317–361, 2001.

[7] Jan A. Bergstra and Jan Willem Klop. Strong normalization and perpetual reductions in the lambda calculus. *Journal of Information Processing and Cybernetics*, 18:403–417, 1982.

[8] Alonzo Church and J. Barkley Rosser. Some properties of conversion. *Transactions of the American Mathematical Society*, 39(3):472–482, 1936.

[9] Thierry Coquand. An analysis of Girard's paradox. In *Proceedings of the First Annual IEEE Symposium on Logic in Computer Science (LICS '86)*, pages 227–236. IEEE Computer Society Press, 1986.

[10] Thierry Coquand. A new paradox in type theory. In *Logic, Methodology and Philosophy of Science IX*, pages 555–570. Elsevier, 1994.

[11] Thierry Coquand and Hugo Herbelin. A-translation and looping combinators in pure type systems. *Journal of Functional Programming*, 4(1):77–88, 1994.

[12] Philippe de Groote. The conservation theorem revisited. In M. Bezem and J. F. Groote, editors, *Typed Lambda Calculi and Applications (TLCA '93)*, volume 664 of *Lecture Notes in Computer Science*, pages 163–178. Springer, 1993.

[13] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[14] Herman Geuvers. The Church–Rosser property for $\beta\eta$-reduction in typed λ-calculi. In *Proceedings of the Seventh Annual IEEE Symposium on Logic in Computer Science (LICS '92)*, pages 453–460, 1992.

[15] Herman Geuvers. Inconsistency of classical logic in type theory, November 2007. Public note.

[16] Herman Geuvers and Mark-Jan Nederhof. Modular proof of strong normalization for the calculus of constructions. *Journal of Functional Programming*, 1(2):155–189, 1991.

[17] Herman Geuvers and Benjamin Werner. On the Church–Rosser property for expressive type systems and its consequences for their metatheoretic study. In *Proceedings of the Ninth Annual IEEE Symposium on Logic in Computer Science (LICS '94)*, pages 320–329. IEEE Computer Society Press, 1994.

[18] Jan Herman Geuvers. *Logics and Type Systems*. PhD thesis, Katholieke Universiteit Nijmegen, 1993.

[19] Neil Ghani. Eta-expansions in dependent type theory: the calculus of constructions. In P. de Groote and J. R. Hindley, editors, *Typed Lambda Calculi and Applications (TLCA '97)*, volume 1210 of *Lecture Notes in Computer Science*, pages 164–180. Springer, 1997.

[20] Jean-Yves Girard. *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*. PhD thesis, Université Paris VII, 1972.

[21] Jean-Yves Girard. *Proofs and Types*. Cambridge University Press, 1989. Translated and with appendices by Paul Taylor and Yves Lafont.

[22] Robert Harper, Furio Honsell, and Gordon Plotkin. A framework for defining logics. *Journal of the ACM*, 40(1):143–184, 1993.

[23] Antonius J. C. Hurkens. A simplification of Girard's paradox. In Mariangiola Dezani-Ciancaglini and Gordon Plotkin, editors, *Typed Lambda Calculi and Applications*, volume 902 of *Lecture Notes in Computer Science*, pages 266–278. Springer, 1995.

[24] Michael Karr. "Delayability" in proofs of strong normalizability in the typed lambda calculus. In H. Ehrig, C. Floyd, M. Nivat, and J. Thatcher, editors, *Mathematical Foundations of Software Development (TAPSOFT '85)*, volume 185 of *Lecture Notes in Computer Science*, pages 208–222. Springer, 1985.

[25] Jeroen Ketema, Jan Willem Klop, and Vincent van Oostrom. Vicious circles in rewriting systems. Artificial Intelligence Preprint Series 52, Utrecht University, 2004.

[26] Assaf J. Kfoury and Joe B. Wells. New notions of reduction and non-semantic proofs of β-strong normalization in typed λ-calculi. In *Proceedings of the Tenth Annual IEEE Symposium on Logic in Computer Science (LICS '95)*, pages 311–321. IEEE Computer Society Press, 1995.

[27] Zurab Khasidashvili. *Form Reduction Systems and Reductions of Contracted Forms and Lambda-Terms*. PhD thesis, Tbilisi State University, 1988. In Russian.

[28] Jan Willem Klop. *Combinatory Reduction Systems*. Number 127 in Mathematical Centre Tracts. Mathematisch Centrum, Amsterdam, 1980. PhD thesis, Utrecht University.

[29] Ralph Loader. Normalisation by translation. Presented at the BRA TYPES workshop, Turin, 1995. Unpublished.

[30] Zhaohui Luo. *An Extended Calculus of Constructions*. PhD thesis, University of Edinburgh, 1990.

[31] Paul-André Melliès and Benjamin Werner. A generic normalisation proof for pure type systems. In E. Giménez and C. Paulin-Mohring, editors, *Types for Proofs and Programs (TYPES '96)*, volume 1512 of *Lecture Notes in Computer Science*, pages 254–276. Springer, 1998.

[32] Nathan Mull. Strong normalization from weak normalization in non-dependent pure type systems via thunkification. Report, University of Chicago, October 2022.

[33] Nathan Mull. An irrelevancy-eliminating translation of pure type systems. In *28th International Conference on Types for Proofs and Programs (TYPES 2022)*, volume 269 of *Leibniz International Proceedings in Informatics*, pages 7:1–7:21. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023.

[34] Robert P. Nederpelt. *Strong Normalization in a Typed Lambda Calculus with Lambda Structured Types*. PhD thesis, Technische Hogeschool Eindhoven, 1973.

[35] OpenAI. Weak and strong normalization in pure type systems. Preprint dated September 25, 2026, with a Lean formalization, September 2026.

[36] Ankit Pradhan, Cole Vick, Ruijie Fang, and Carl Kwan. PTS-beta-eta: weak and strong $\beta\eta$-normalization in pure type systems with $\beta\eta$-conversion. Lean 4 formalization with Mathlib, Lean `v4.35.0`, 2026.

[37] Cody Roux. Internal proofs of strong normalization. TYPES 2025, extended abstract and presentation, June 2025. Slides: `https://msp.cis.strath.ac.uk/types2025/slides/TYPES2025-slides54.pdf`.

[38] Cody Roux and Floris van Doorn. The structural theory of pure type systems. In Gilles Dowek, editor, *Rewriting and Typed Lambda Calculi*, volume 8560 of *Lecture Notes in Computer Science*, pages 364–378. Springer, 2014.

[39] M. H. B. Sørensen. *Normalization in λ-Calculus and Type Theory*. PhD thesis, University of Copenhagen, 1997. DIKU Report 97/27.

[40] Morten Heine Sørensen. Problem #9: Is every weakly normalizing PTS also strongly normalizing? TLCA List of Open Problems, 1993. Submitted in 1993. Problem posed by Henk Barendregt, Herman Geuvers and Jan Willem Klop.

[41] Morten Heine Sørensen. Strong normalization from weak normalization in typed λ-calculi. *Information and Computation*, 133(1):35–71, 1997.

[42] William W. Tait. Intensional interpretations of functionals of finite type I. *Journal of Symbolic Logic*, 32(2):198–212, 1967.

[43] Masako Takahashi. Parallel reductions in λ-calculus. *Information and Computation*, 118(1):120–127, 1995.

[44] Alfred Tarski. A lattice-theoretical fixpoint theorem and its applications. *Pacific Journal of Mathematics*, 5(2):285–309, 1955.

[45] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020.

[46] The Rocq Development Team. Hurkens's paradox, file `Logic/Hurkens.v`. Rocq standard library.

[47] Femke van Raamsdonk, Paula Severi, Morten Heine Sørensen, and Hongwei Xi. Perpetual reductions in λ-calculus. *Information and Computation*, 149(2):173–225, 1999.

[48] Hongwei Xi. Weak and strong beta normalisations in typed λ-calculi. In P. de Groote and J. R. Hindley, editors, *Typed Lambda Calculi and Applications (TLCA '97)*, volume 1210 of *Lecture Notes in Computer Science*, pages 390–404. Springer, 1997.